

Reviewer Pack — Baker-Norine ω -Gonality Lower-Bounds Tseitin DPLL Size

Entry 9dfcbea8b0a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 19:38:49 UTC

Baker-Norine ω -Gonality Lower-Bounds Tseitin DPLL Size

Entry ID: 9dfcbea8b0a8 Verdict: INCONCLUSIVE Recorded: 2026-05-18 19:38:49 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Riemann-Roch / chip-firing divisor theory on finite graphs (Baker-Norine 2007 ‘Riemann-Roch and Abel-Jacobi theory on a finite graph’; Hladký-Král’-Norine 2013 ‘Rank of divisors on tropical curves’; Cools-Draisma-Payne-Robeva 2012 tropical Brill-Noether; Caporaso 2014). The Baker-Norine rank $r_{\text{BN}}(D)$ of a divisor $D \in \mathbb{Z}^V$ uses Dhar’s burning algorithm — a parallel chip-firing reduction with set-firing on subsets $S \subseteq V \setminus \{\text{base}\}$ satisfying $D(v) \geq \deg_S(v)$; the rank test for $r \geq 1$ enumerates ‘ $D - v \sim \text{effective}$ ’ for every $v \in V$. The functional uses integer subtraction, max/argmax over firing sets, and the order predicate $D \geq 0$ (non-ring; there is no $\mathbb{F}_q$ polynomial extension since ‘ ≥ 0 ’ and base-reduced canonical form are not preserved under polynomial ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe. ArXiv ‘Baker-Norine’ AND (‘Tseitin’ OR ‘resolution refutation’ OR ‘proof complexity’) returns 0 direct hits with <5 adjacent papers (all on tropical Brill-Noether, never on CNF refutation). Distinct from blacklisted 2-Sylow rank of $K(G)$ (a property of G alone, here we use rank of a (G, ω) -specific divisor), hook-length dim of T-join partition (representation-theoretic dimension of a Specht module), signed-Laplacian holonomy / effective resistance / $p=3$ modulus / Szegedy phase gap / Fiedler IPR (spectral/metric), fatgraph genus (oriented surface embedding), and dismantlability core (closed-neighborhood domination order) — none uses Baker-Norine divisor rank of the charge divisor.

Field B (complexity object): Tree-like Resolution / DPLL refutation size $t(T(G, \omega))$ for Tseitin XOR formulas $T(G, \omega)$ on connected 3-regular graphs G with odd charge $\omega: V \rightarrow \mathbb{F}_2$ (Urquhart 1987; Ben-Sasson-Wigderson 2001; Alekhnovich-Razborov 2001 expansion regime), in the Cook-Reckhow-Krajíček bounded-arithmetic framework where $\neg T(G, \omega)$ is Σ^b_0 and $\log_2 t(T(G, \omega))$ lower-bounds V^0 -witnessing complexity. Tree-Resolution size is the canonical Frege-depth stepping stone via Pudlák-Buss feasible interpolation; Tseitin lower bounds for Extended Frege remain an open target.

Statement:

Let G be a connected 3-regular graph on n vertices with odd charge $\omega: V \rightarrow \mathbb{F}_2$ ($\Sigma\omega \equiv 1 \pmod 2$), and let $T = \omega^{-1}(1)$. Define the ω -gonality $\rho(G, \omega) := \min\{\deg(D) : D \in \mathbb{Z}^V, D \geq 0, \text{supp}(D) \subseteq T, r_{\text{BN}}(D) \geq 1\}$, where r_{BN} is the Baker-Norine rank computed via Dhar’s burning algorithm (D has rank ≥ 1 iff $D - v$ is linearly equivalent to an effective divisor for every $v \in V$). Then for every such (G, ω) with $n \geq 8$, $\log_2 t(T(G, \omega)) \geq \rho(G, \omega) / 4$, where $t(T(G, \omega))$ is the smallest tree-like Resolution refutation size of the Tseitin XOR CNF.

Rationale (proposer’s reasoning):

The Baker-Norine rank of the charge divisor measures whether the odd-vertex set T can support a non-trivial linear system on the chip-firing Jacobian of G — a global divisor-

theoretic obstruction analogous to the parity obstruction that forces Tseitin to be unsatisfiable, but quantitative rather than binary. High ω -gonality means no small effective divisor on T ‘moves’ under any single chip removal, witnessing a structural rigidity of (G, ω) that should propagate to refutation difficulty via the Ben-Sasson-Wigderson width-vs-expansion lemma applied to the chip-firing-induced cycle-space decomposition. The invariant is structural on (G, ω) (two formulas with identical constant-FALSE truth tables have different ρ), shielding from Razborov-Rudich; Dhar’s burning uses non-ring max/order operations safe from Aaronson-Wigderson.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eb23cc2c77664282

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 3 sizes $n \in \{8, 10, 12\} = 90$ trials, for each (G, ω) compute $\rho(G, \omega)$ via Baker-Norine/Dhar and $t(T(G, \omega))$ via *tree-DPLL (cap $2e5$, censored upward)*. *Conjecture is SUPPORTED iff zero trials violate $\rho/4 > \log_2 t$ AND Spearman $\rho_S(\log_2 t^*, \rho) \geq 0.4$; FALSIFIED if any trial violates the inequality.*

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Baker-Norine gonality Tseitin resolution proof complexity - chip-firing divisor rank tree resolution lower bound - graph gonality Dhar burning algorithm DPLL Tseitin

Top relevant hits considered: - [http://arxiv.org/abs/1504.06713v2] Computing graph gonality is hard - [http://arxiv.org/abs/2010.07992v2] Ternary and Quaternary Curves of Small Fixed Genus and Gonality with Many Rational Points - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1407.6958v3] Chip-firing games on Eulerian digraphs and NP-hardness of computing the rank of a divisor on a graph - [http://arxiv.org/abs/0811.2800v2] Parallel Chip-Firing on the Complete Graph: Devil’s Staircase and Poincare Rotation Number - [http://arxiv.org/abs/2002.07753v2] Gonality sequences of graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def matrix_mult(A, B):
    return [[sum(a * b for a, b in zip(row, col)) for col in zip(*B)] for row in A]

def matrix_add(A, B):
    return [[a + b for a, b in zip(row_a, row_b)] for row_a, row_b in zip(A, B)]

def matrix_sub(A, B):
    return [[a - b for a, b in zip(row_a, row_b)] for row_a, row_b in zip(A, B)]

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_rank(A):
    if not A:
        return 0
    rank = 0
    for col in range(len(A[0])):
        pivot = -1
        for row in range(rank, len(A)):
            if A[row][col] != 0:
                pivot = row
                break
        if pivot == -1:
            continue
        A[rank], A[pivot] = A[pivot], A[rank]
        for row in range(rank + 1, len(A)):
            if A[row][col] != 0:
                factor = Fraction(A[row][col], A[rank][col])
                A[row] = [a - factor * b for a, b in zip(A[row], A[rank])]
        rank += 1
    return rank

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graphs")
    edges = []
    stubs = list(range(n)) * 3
    while stubs:
        u = random.choice(stubs)
        stubs.remove(u)
        v = random.choice([x for x in stubs if x != u])
        stubs.remove(v)
        edges.append((u, v))
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    visited = [False] * n
```

```

stack = [0]
visited[0] = True
while stack:
    u = stack.pop()
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            stack.append(v)
if not all(visited):
    return generate_3_regular_graph(n, seed + 1)
return adj

def generate_odd_charge(n, seed):
    random.seed(seed)
    omega = [random.randint(0, 1) for _ in range(n)]
    if sum(omega) % 2 == 0:
        omega[0] = 1 - omega[0]
    return omega

def compute_t_star(adj, omega, max_calls=200000):
    n = len(adj)
    T = [i for i, o in enumerate(omega) if o == 1]
    clauses = []
    for u in range(n):
        neighbors = adj[u]
        clause = []
        for v in neighbors:
            clause.append((u, v))
        clauses.append(clause)
    call_count = 0
    def dpll(clauses, assignment):
        nonlocal call_count
        call_count += 1
        if call_count > max_calls:
            return float('inf')
        if not clauses:
            return 1
        for clause in clauses:
            if all((lit in assignment) and (assignment[lit] == False) for lit in clause):
                return 0
        unit_clauses = [c for c in clauses if len(c) == 1]
        while unit_clauses:
            lit = unit_clauses[0][0]
            assignment[lit] = True
            new_clauses = []
            for clause in clauses:
                if lit not in clause:
                    new_clause = [l for l in clause if l != (-lit,)]
                    if not new_clause:
                        return 0
                    new_clauses.append(new_clause)
            clauses = new_clauses
            unit_clauses = [c for c in clauses if len(c) == 1]
        if not clauses:
            return 1
        lit = clauses[0][0]

```

```

        return dpll([c for c in clauses if lit not in c], {**assignment, lit: True}) + \
            dpll([c for c in clauses if (-lit,) not in c], {**assignment, lit: False})
t_star = dpll(clauses, {})
return min(t_star, max_calls)

```

```

def compute_rho(adj, omega):
    n = len(adj)
    T = [i for i, o in enumerate(omega) if o == 1]
    g = len([(u, v) for u in range(n) for v in adj[u] if u < v]) - n + 1
    for deg in range(1, g + 2):
        for D in itertools.combinations(T, deg):
            D = list(D)
            for v in range(n):
                D_minus_v = [d for d in D if d != v]
                if len(D_minus_v) == len(D):
                    continue
                rank = dhar_burning(adj, D_minus_v)
                if rank >= 1:
                    return deg
    return g + 1

```

```

def dhar_burning(adj, D):
    n = len(adj)
    A = [[0] * n for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            A[u][v] = 1
    for d in D:
        A[d][d] = 1
    return matrix_rank(A)

```

```

def run_trial(seed):
    random.seed(seed)
    n = random.choice([8, 10, 12])
    adj = generate_3_regular_graph(n, seed)
    omega = generate_odd_charge(n, seed)
    t_star = compute_t_star(adj, omega)
    rho = compute_rho(adj, omega)
    if t_star <= 0:
        return {
            "metric_name": "log2_t_star_vs_rho",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"t_star={t_star} <= 0 for n={n}"
        }
    log2_t_star = math.log2(t_star)
    metric_value = log2_t_star - rho / 4
    conjecture_holds = metric_value >= 0
    counterexample = "" if conjecture_holds else f"rho/4={rho/4} > log2_t_star={log2_t_star} for n={n}"
    return {
        "metric_name": "log2_t_star_vs_rho",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        trials.append(trial)
    metric_values = [t["metric_value"] for t in trials]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(t["conjecture_holds"] for t in trials) / len(trials)
    if all(t["conjecture_holds"] for t in trials):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(t["seed"] for t in trials if not t["conjecture_holds"])
        counterexample = next(t["counterexample"] for t in trials if not t["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 184, in <module>
    trial = run_trial(seed)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 158, in run_trial
    t_star = compute_t_star(adj, omega)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 124, in compute_t_star
    t_star = dpll(clauses, {})
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 122, in dpll
    return dpll([c for c in clauses if lit not in c], {**assignment, lit: True}) + \
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 122, in dpll
    return dpll([c for c in clauses if lit not in c], {**assignment, lit: True}) + \
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 122, in dpll
    return dpll([c for c in clauses if lit not in c], {**assignment, lit: True}) + \
            ~~~~~
  [Previous line repeated 6 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1474ec5.py", line 123, in dpll
    dpll([c for c in clauses if (-lit,) not in c], {**assignment, lit: False})
            ~~~~~
TypeError: bad operand type for unary -: 'tuple'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` ('bad operand type for unary -: tuple') inside the DPLL routine before any trials produced data, so neither the violation count nor the Spearman correlation could be evaluated. | next: Fix the DPLL implementation to represent clauses as sets/lists of signed integer literals (not tuples) so that negation works, then re-run the 90-trial protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	342290
2	preregistration	claude_max	opus	0	0	6399
3	novelty	claude_max	opus	0	0	3354
4	novelty	claude_max	opus	0	0	9790
5	test_gen	mistral	codestral-latest	0	0	316716
6	test_gen	mistral	codestral-latest	0	0	324161
7	test_gen	mistral	codestral-latest	0	0	324214
8	test_gen	mistral	codestral-latest	0	0	318709
9	judge	claude_max	opus	0	0	4693

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1650326 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/9dfcbea8b0a8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9dfcbea8b0a8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9dfcbea8b0a8.tar.gz` (if generated)